

Authentication System Design (High Load)

Scenario: Your user base exploded from 50k to 3 million monthly active users in six months. Logins are now taking 3–5 seconds, and authentication servers are getting overloaded during peak hours. Design an end-to-end login system that can handle this load reliably.

A: The slow login time points to a **database bottleneck** (password verification) and **high transaction costs** (creating sessions). I would shift to a fast, stateless system.

Component	Design Choice	Why It Solves the Problem
Authentication Flow	Stateless JWT (JSON Web Tokens)	The server verifies the token's signature, avoiding slow database lookups for session data on every single request.
Password Storage	BCrypt / SCrypt with dedicated read replicas	Use secure hashing (BCrypt) but route all verification requests to fast, read-only database replicas. This offloads the slow write master database.
Load Balancer	External Load Balancer (ALB/NGINX)	Distributes incoming traffic evenly across a large pool of authentication service instances, preventing any single server from overloading.
Caching	Distributed Cache (Redis) for Rate Limiting	Store login attempt counts per user/IP in Redis (which is extremely fast). This stops brute-force attacks and protects the database from excessive failed login attempts during peak spikes.
Session Storage	Stateless Token (JWT)	The authentication status is stored in the token on the client's side, eliminating the need for the server to store or look up millions of session records in the database.
Resilience	Circuit Breaker (on external services)	Implement a Circuit Breaker (Resilience4J) on any external services the auth flow depends on (e.g., email verification). If that external service slows down, the auth service fails fast instead of getting stuck, avoiding cascading failure.

Real-Time Notification System (Uber/IoT)

Scenario: You are asked to design a real-time notification system for a ridesharing app like Uber. Drivers and customers must receive updates instantly. Describe the entire architecture including WebSockets/SSE, message brokers, scaling strategy, ordering guarantees, retries, and how you prevent message loss.

A: This system requires a central message backbone for high throughput and WebSockets for real-time client delivery.

Component	Design Choice	Why It Solves the Problem
Messaging Core	Kafka (as a message broker)	Kafka handles millions of events per second with high durability. It acts as the central source of truth for all events (e.g., driver_location_update).
Real-time Delivery	WebSockets (or SSE)	WebSockets provide a full, persistent, bidirectional connection between the server and the mobile app, guaranteeing instant, low-latency updates.
Architecture	Spring Boot WebSockets Gateway	A cluster of small, specialized Spring Boot services listens to Kafka. When an event for a user arrives, the service forwards the message instantly to the correct client via the established WebSocket connection.
Scaling	Partitioning and Horizontal Scaling	Kafka topics are partitioned based on userId or rideId. Services are scaled horizontally (many instances) where each instance consumes a subset of partitions, ensuring load distribution.
Message Loss	Durable Storage (acks=all)	Kafka producers require full acknowledgment (acks=all) before considering a message sent. Consumers must use manual offset commits after successful processing to prevent losing messages during a crash.
Ordering/Retries	Keyed Partitioning and DLTs	Guarantee order by ensuring all events for one ride are sent to the same Kafka partition (using rideId as the

Component	Design Choice	Why It Solves the Problem
		key). Failed messages are retried and eventually moved to a Dead Letter Topic (DLT) for separate analysis.

Product Search System (Amazon Scale)

Scenario: Your company wants to build a product search system similar to Amazon. The catalog has 25 million products and updates happen continuously. Search results must appear within 100 ms. Explain how you would design the indexing pipeline, search engine, cache layers, shard strategy, and how you handle updates without affecting performance.

A: This requires shifting the read workload from the database to a specialized, fast search engine.

Component	Design Choice	Why It Solves the Problem
Search Engine	Elasticsearch or Apache Solr	These are built on Lucene and are designed specifically for fast, full-text search and indexing, achieving sub-100ms latency on massive datasets.
Indexing Pipeline	Kafka + Worker Service	The database change events are streamed into a Kafka topic . A dedicated worker microservice consumes this topic and asynchronously pushes the changes to the search engine. This ensures the database write transaction is never slowed down by the search index update.
Sharding Strategy	Index Sharding (by Product Category or Region)	Products are split into multiple, smaller indices (shards). Queries run against all shards in parallel. Sharding is essential for distributing the 25 million documents across multiple nodes and scaling horizontally.
Caching Layer	Query Caching (in Elasticsearch) and Redis	Elasticsearch internally caches frequently run query results. Redis is used for caching popular product metadata or entire search results for the top few hundred queries, reducing load on the search cluster.

Component	Design Choice	Why It Solves the Problem
Handling Updates	Eventually Consistent Indexing	Updates are handled asynchronously via the Kafka pipeline. This allows the primary database to commit immediately. The small delay (seconds) before the change appears in the search index is an acceptable trade-off for continuous performance.

SCALABILITY : Detailed Scenario Questions

4. Your microservice handles profile updates and receives around 5–7k requests per second during peak time. Suddenly, latency jumps from 150 ms to 3 seconds. CPU usage is normal but DB read queries spike. Explain how you would identify the bottleneck, what metrics you would check, how you would scale the system horizontally/vertically, and what architectural changes you would introduce to prevent this happening again.

A: The bottleneck is most likely the **database read load** or an **unoptimized query** that triggers slow disk I/O.

- **Identify Bottleneck:**
 - **Metrics Check:** Immediately check the **Database Connection Pool Saturation** and **Slow Query Logs** (time taken per query). Since CPU is normal, the thread is likely **blocked** waiting for the database, causing latency.
 - **Diagnosis:** The spike in read queries indicates an inefficient operation, likely an **N+1 SELECT problem** or a query performing a **full table scan** due to a missing index.
- **Scaling and Fix:**
 - **Immediate Scale:** Scale the **database vertically** (more RAM/CPU) as a temporary fix, and scale the **application horizontally** (more instances/pods) to handle connection load.
 - **Architectural Fix:** Implement a **caching layer (Redis)** for frequently read user profile data, drastically reducing database load. Analyze and add the missing **database index** to the slow query, or refactor the Hibernate/JPA code to use **@EntityGraph** to prevent the N+1 problem.

5. A Kafka consumer service in your system is processing messages much slower than the rate at which they are produced. Lag grows continuously and sometimes takes hours to catch up. The data is time-sensitive. Walk me through how you would analyze partitioning, consumer concurrency, batch sizes, DB writes, and backpressure handling to make this system scalable.

A: The problem is the consumer's inability to keep pace with the producer (processing latency).

- **Analyze & Tune:**
 - **Partitions & Concurrency:** I would verify that the **number of consumers equals the number of partitions** to maximize parallel processing. If the partitions are insufficient, I would increase them (and the consumer count).
 - **Batch Sizes:** I would tune the consumer's **max.poll.records** to pull large batches of messages, amortizing the overhead of network calls and transaction commits.
 - **DB Writes:** If processing involves heavy DB writes, I would switch from individual writes to **JDBC batching** (bulk inserts/updates) to reduce transaction overhead and speed up persistence.
- **Backpressure/Scalability:**
 - If the lag is due to a slow external service, I would offload that part to a separate **async service** or implement **backpressure** logic to slow down the producer (if the whole system is owned). Ultimately, the consumers need more CPU/resources to process the data faster.

TRADE-OFFS : Detailed Scenario Questions

6. Your team needs to store analytics events generated by millions of users every hour. The product team wants flexible querying, but the infra team wants minimal cost, and the DB team wants high write throughput. You must pick between SQL, NoSQL, columnar stores, and time-series databases. Explain which one you choose, the trade-offs in consistency, cost, performance, data modeling, and operational overhead.

A: I would choose a **Columnar Store** (e.g., ClickHouse, or cloud-native solutions like AWS Redshift/Snowflake).

Factor	Columnar Store (ClickHouse)	Trade-Off Explanation
Consistency	Eventual/Low Consistency	Updates are not critical (analytics). We accept eventual consistency for massive write speed.
Cost & Throughput	High Write Throughput, Minimal Storage Cost	Columnar storage minimizes I/O by only reading necessary columns, making it extremely efficient for large-volume writes and complex read queries.
Performance	Fastest Querying for Aggregates	Excellent for flexible, aggregate queries (SUM, AVG, COUNT) across millions of records, meeting the 100ms requirement better than traditional row-based SQL.
Data Modeling	Rigid/Denormalized Schema	The schema is optimized for reading, meaning data modeling is less flexible than NoSQL and must be carefully planned upfront.
Operational Overhead	High	Operating a columnar cluster requires specialized knowledge for sharding, replication, and query optimization, increasing operational complexity.

7. Your CTO asks whether the company should move from a single Spring Boot monolith to microservices. You have a team of 6 developers and limited DevOps support. Describe what factors you would evaluate, what trade-offs exist, where microservices might cause more harm than good, and whether you would recommend migrating or not.

A: I would **not recommend migrating** immediately.

- **Evaluation Factors:** I would check the current monolith's complexity, its deployment time, and whether different parts of the application require different scaling rates.
- **Trade-offs (Harmful Side):** Moving to microservices introduces **massive operational overhead** (managing 10+ services, network calls, distributed logging, monitoring, and debugging). With only 6 developers and limited DevOps support, the team would spend more time maintaining infrastructure than delivering features.
- **Recommendation:** I would recommend moving to a **Modular Monolith** first. This architecture uses clean interfaces and packages internally, achieving the benefit of

organized code and separation of concerns **without** the operational complexity of distributed network communication and managing dozens of Docker containers.

DEBUGGING PRODUCTION ISSUES : Detailed Scenario Questions

8. During peak load, one of your payment microservices starts throwing intermittent 504 Gateway Timeout errors, but only for 8–10% of requests. CPU and memory look normal, but logs show occasional slow DB queries. Walk me through every step you would take to debug this—from logs, metrics, tracing, thread dumps, DB profiling—and how you'd prevent similar incidents in the future.

A: The intermittent 504 errors (Gateway Timeout) suggest threads are blocked waiting for a response, likely from the database, despite normal CPU usage.

1. **Metrics & Tracing:** I check **Latency Metrics** at the payment service and the database. I analyze **Distributed Traces (Jaeger/Zipkin)** to confirm the failing requests are spending 99% of their time waiting on a single **database span**. I also check the **Connection Pool** health to see if connections are exhausted.
2. **Thread Dump:** I immediately capture a **jstack thread dump** during the failure window. I look for threads in the **TIMED_WAITING** or **BLOCKED** state with a stack trace pointing to JDBC connection calls, confirming a database bottleneck.
3. **DB Profiling:** I check the **Slow Query Log** on the database server. If the query is complex, I run an **EXPLAIN ANALYZE** to find a missing index or an inefficient join causing the occasional spike.
4. **Prevention:** I would implement a **read-timeout** on the database client to prevent threads from waiting 30 seconds for a failed query. I would also apply the missing **database index** and review the connection pool settings.

9. A memory leak slowly increases heap usage across multiple pods in production. After ~3 hours, the pods crash and Kubernetes restarts them. Describe your exact debugging playbook: what logs you check, how you capture heap dumps, what tools you use to analyze them, how you identify the leak, and how you validate the fix before redeployment.

A: The slow memory growth followed by a crash suggests a classic **memory leak** (unreleased object references).

1. **Logs & Metrics:** Check service logs for **OutOfMemoryError: Java heap space** errors. Review metrics (Prometheus/Grafana) to verify that **Garbage Collection (GC)** activity is high, but the heap usage after GC is increasing over time (the "sawtooth pattern").
2. **Capture Heap Dump:** I capture a heap dump using a tool like `jcmd <PID> GC.heap_dump` on a running pod just before it crashes (e.g., at 90% memory utilization).
3. **Analysis:** I load the dump into **Eclipse Memory Analyzer Tool (MAT)**. I use the "Dominator Tree" and "Leak Suspects" report to find the largest objects that are still being referenced (preventing GC), such as a massive, unintended **static HashMap** or a list holding session data.
4. **Validation:** The fix is validated by ensuring that the memory usage stabilizes in the **QA environment** over a long stress test (e.g., 24 hours).

OWNERSHIP & ARCHITECTURE : Detailed Scenario Questions

10. You lead a team managing a core microservice that other 12 services depend on. A change from your team caused a production outage and broke downstream services. Explain how you would handle the next 2 hours: rollback, communication with other teams, triage, root-cause analysis, writing post-mortem, introducing safeguards (like feature flags, contract tests, canary releases), and preventing this in the future. Show full ownership.

A: I would take immediate, decisive action focusing on restoration and transparency.

1. **Immediate Rollback (0–15 mins):** Execute an **immediate, low-risk rollback** to the last known stable version of my core service. Verification is simple: confirm downstream services stop reporting errors.
2. **Communication & Triage (15–30 mins):** Send a clear, concise "**Incident Alert**" to all impacted teams and leadership, stating the service, the confirmed issue (e.g., "Outage due to recent deployment"), and the **rollback timeline**. Create a central communication channel (Slack/bridge) for updates.

3. **Triage & Root Cause (30–120 mins):** Analyze **Distributed Traces** (Jaeger/Zipkin) to isolate the single line of code or configuration change that caused the failure (e.g., a missing database index or a faulty cache eviction).
4. **Safeguards & Prevention:**
 - **Feature Flags:** Commit the faulty change *behind a feature flag* immediately so it can be tested safely after the fix.
 - **Contract Tests:** Enforce **Consumer-Driven Contract (CDC) tests (Pact)** to ensure future changes do not break downstream dependencies.
 - **Deployment Strategy:** Switch to **Canary Releases**, exposing future deployments to only 5% of traffic before full rollout.
 - **Post-Mortem:** Write a draft post-mortem summarizing the root cause, actions taken, and the new safeguards put in place, emphasizing transparent ownership of the error.